With the rise of HTML5, graphics and visualisation have become more informative and interactive. DSLs are playing a key role in creating rich and interactive visualisation.

Processing

Processing, earlier referred to as P5 or Processing, is a domain-specific programming language for designing interactive graphics and animated images, and for manipulating media files. It is highly useful for UI designers, artists and researchers to quickly explain their thoughts in a more visual way.

The Processing Development Environment (PDE)

The processing development environment is a handy IDE for creating, compiling and executing processing programs. PDE is a text editor with menus for operations like compiling, running, etc, and a console area for printing error messages and execution statuses.

A program written using the Processing language is referred to as a 'sketch' and the whole program/project folder that consists of sketches and other resources is referred to as a 'sketchbook'. Since processing adopts its programming style from Java, we can use any Java IDE to develop programs.

The package, Processing, is available for download from *https://processing.org/download*. Installation steps include downloading the proper Zip, based on the target operating system, and extracting it in a local directory. There is a wide range of built-in sample projects that explain the different features, and they can be accessed through Examples options in file menu in PDE.

Modes in Processing

Sketches drawn using Processing can be exported and used in different environments. Based on the target environment, we can choose the modes in PDE. The following are the different modes available in PDE:

- *Java:* By default, PDE runs in Java mode and output can be executed in any operating system such as Linux, Mac or Windows. Also, any Java class can be imported into PDE and re-used inside the sketches.
- *JavaScript:* You can add or switch to JavaScript mode in PDE. JavaScript mode leverages the *processing.js* library to export the sketch to run in the Web environment using HTML5 and WebGL. This provides the ability to run the sketches in a browser without any plug-in dependency.
- *Android:* By adding Android mode, you can run the sketches in an Android environment. By installing and configuring the SDK and emulator, you can integrate it with PDE and run the sketches in it.

Based on the selected mode, sketches can be exported as a fully contained application in the respective language. For example, in Java mode, PDE exports the sketch as an applet with all its dependencies. Similarly, in JavaScript mode, it is exported as a JS file.

Figure 1: Processing PDE

Getting started with Processing

Java developers would find it easy to use Processing as the programming style matches Java. Most of the object-oriented concepts can be used. There are two special functions in Processing, which are called internally:

- *setup():* This function will be executed only once during the initial start of the sketch. Initial screen layout, UI initialisation and other basic configurations are done in this function.
- *draw():* This will be called automatically by Processing for each frame and it renders graphical animation.

Apart from these two functions, there are many in-built core functions and event handlers to handle user actions and to design various shapes. All functions in Processing are straightforward and the names are self-explanatory. Refer to *http://processing.org/reference/* for a detailed list of functions and their documentation, along with examples.

Static sketch – 'Hello World!': In this example, let's look at how to draw a static sketch. In Processing, you need X-Y pixel coordinates to draw the shapes. For example, to draw a line, you need two points with X-Y coordinates. Similarly, to draw a rectangle or ellipse, you need four points with X-Y coordinates.

```
size(400,400);
int widthLen = width/2;
int heightLen = height/2-100;
rectMode(CENTER);
rect(widthLen, heightLen + 75, 60, 100);
line(widthLen,heightLen+20, widthLen+80, heightLen+80);
line(widthLen,heightLen+20, widthLen-80, heightLen+80);
ellipse(widthLen-55,heightLen-5,12,25);
ellipse(widthLen+55,heightLen-5,12,25);
ellipse(widthLen,heightLen,100,100);
ellipse(widthLen-25,heightLen-7,25,12);
```